

TimeSync: Environmental Event Sensing for Time Synchronization across heterogeneous IoT devices

MANAV DHELIA* and FURQUAAN SYED*, University of Massachusetts Amherst, USA

GitHub: <https://github.com/manavdhelia/Time-Synchronization-Via-Sensing>

Time synchronization among resource-constrained IoT devices is considered a crucial but challenging process due to the inherent limitation in computation, energy, and bandwidth. Our system, TimeSync, offers a novel solution for effectively attaining time synchronization by exploring environmental event sensing. Unlike traditional schemes that depend on dedicated protocols for device-device synchronization, TimeSync takes advantage of ambient events observed by heterogeneous sensors such as IMU, pressure, and ambient light sensors to perform clock alignment among devices. Our evaluations show that TimeSync achieves submillisecond clock synchronization accuracy in dynamic environments with low computational overhead. Using commodity hardware and ubiquitous sensors, TimeSync proves its applicability in diverse IoT ecosystems, from smart homes to wearable networks. This approach not only saves energy, but also ensures robust and scalable synchronization, paving the way for reliable coordination in IoT systems.

Additional Key Words and Phrases: Time Synchronization, Internet of Things, Ambient Event Sensing

ACM Reference Format:

Manav Dhelia and Furqaan Syed. 2018. TimeSync: Environmental Event Sensing for Time Synchronization across heterogeneous IoT devices. *ACM Trans. Graph.* 37, 4, Article 111 (August 2018), 8 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Precise temporal alignment on heterogeneous and distributed devices in a network, without additional communication overhead in resource-constrained devices, has been an important research issue. Synchronization of time is critical for a variety of applications, such as distributed sensing, swarm robotics, and networked control systems. Reliable temporal alignment enables smooth operation, optimal resource utilization, and reliable data aggregation in these applications. Synchronization with low latency and energy consumption becomes more complicated as a result of the increasing heterogeneity of networks, including devices with different capabilities, sensors, and communication protocols. [Noh et al. 2008] Traditional methods are mostly incompetent to deal with such challenges in resource-constrained environments. Our system, TimeSync, overcomes these limitations by leveraging the existing

sensing infrastructure to use ambient events for synchronization. Reduce communication overhead and achieve low-latency synchronization by detecting and timestamping environmental events using sensors such as IMU, pressure, and ambient light. This would improve overall performance in dynamic and heterogeneous IoT networks.

Time synchronization has real-world applications in many domains. In smart homes, synchronized IoT devices enable seamless automation by aligning data from sensors such as motion detectors and light controllers. For example, a motion sensor can trigger a light switch if timestamps align accurately, preventing latency or missed events. According to some studies, time misalignment in smart home networks can lead to up to 20% failure in event-based automation tasks. This synchronization ensures the reliable interaction of devices and enhances user experience while reducing energy wastage. In healthcare, synchronized time-stamps are crucial for wearable devices monitoring physiological parameters such as heart rate and movement patterns. Even small time drifts between devices may result in wrong diagnosis or loss of critical events in wireless body area networks (WBANs). For instance, precise synchronization ensures the trustworthy fusion of information coming from various sensors for a holistic view of the health status of a patient. This is particularly crucial in applications involving remote patient monitoring, where accuracy in data directly influences decision-making. Precise synchronization ensures reliable data fusion across devices.

Traditional time synchronization approaches are often based on dedicated protocols, such as Network Time Protocol or Precision Time Protocol. [Amundson et al. 2007] While these protocols have been widely used to attain synchronized clocks in networks, mainly in industrial and high-performance scenarios, they are not efficient for resource-constrained IoT devices due to the significant imposed communication overhead as depicted in Table 1. NTP, though widely

Protocol	Latency	Overhead	Suitability
NTP	~10ms	High	Poor
PTP	~1μs	Moderate	Moderate
Time Sync Protocol	~100μs	Low	Better

Table 1. Comparison of Time Synchronization Protocols

deployed, imposes very high latency and needs a big amount of data exchange; hence, it is badly fit for IoT contexts. The PTP reaches higher precision with low latency but keeps the energy-consuming and complex nature in the low-power and distributed network implementations. These issues are mitigated by Lightweight Time Synchronization Protocols through reducing data overhead and increasing scalability, but each of these still relies on periodic communications that further limit the applications in dynamic IoT ecosystems. Traditional methods also failed to handle heterogeneous IoT

*Both authors contributed equally to this research.

Authors' Contact Information: Manav Dhelia, mdhelia@umass.edu; Furqaan Syed, fsyed@umass.edu, University of Massachusetts Amherst, Amherst, MA, USA
GitHub: <https://github.com/manavdhelia/Time-Synchronization-Via-Sensing>.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7368/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

network challenges where devices with diversified capabilities and communication protocols were operating. Such environments often suffer from clock drift, further exacerbating the challenge of synchronization by demanding updates at a high frequency, hence high energy and bandwidth consumption. [Coviello et al. 2021] This forms a bottleneck for scenarios requiring real-time coordination, like smart homes or wearable networks, where continuous interaction between devices is an integral requirement. There is thus a very significant demand for an approach to synchronization that keeps the communication overhead low while guaranteeing low latency and accuracy. Addressing these challenges, our proposed system, TimeSync, leverages ambient sensing events through existing infrastructure. TimeSync avoids continuous communication protocols by using environmental events detected via IMU, pressure, and ambient light sensors. By timestamping these events locally and aligning clocks based on shared occurrences, TimeSync reduces energy consumption and communication latency by a wide margin. This innovative approach enables not only robust synchronization in dynamic IoT environments but also enhances scalability and adaptability across diverse applications, making it unique compared to traditional synchronization methods.

The technical specifications focus on the following three main performance metrics:

1. **Synchronization Accuracy:** TimeSync achieves sub-millisecond accuracy, aligning device clocks based on shared events to achieve accurate coordination of real-world scenarios, such as smart homes and wearable networks.

2. **Communication Overhead:** Since TimeSync transmits only events with timestamps, it minimizes data exchange over the air, considerably reducing bandwidth usage compared to traditional approaches like NTP and PTP.

3. **Energy Efficiency:** The system is energy-efficient by using lightweight event detection algorithms and minimizing the frequency of communication, making it suitable for IoT devices that operate on batteries.

TimeSync's architecture leverages multimodal sensing to enhance robustness and adaptability, allowing it to scale across diverse IoT networks while maintaining low latency and high reliability. With TimeSync, addressing the critical challenges of synchronization accuracy, communication overhead, and energy efficiency raises the bar for time synchronization in IoT ecosystems. The idea here is that TimeSync makes use of already available ambient sensors, like IMU, pressure, and ambient light sensors, which can identify and timestamp prominent events. These events would then serve as synchronization signals and eliminate the use of always-on communication protocols, thus reducing energy consumption. The system is especially applicable in dynamic environments where classical synchronization methods cannot work appropriately due to latency and bandwidth limitations.

Fig 1 demonstrates the setting of a smart home with an IMU sensor, a pressure sensor, and an ambient light sensor that are installed in different rooms and have a similar ambient event like a door opening-exhibiting detectable changes across the covered modalities. Timesync utilizes these modalities of sensors positioned strategically in achieving time-synchronized functionality within distributed devices through efficient resources. [Du et al. 2008] The

mechanism of TimeSync is purely receiver-to-receiver (RX-to-RX) synchronization, which means that the devices witness an ambient event in common and timestamp the event independently. For example, the IMU sensor detects a sudden change in acceleration, the pressure sensor registers the change in air pressure caused by the movement of the door, and the ambient light sensor captures changes in light levels as the door opens or closes. Each sensor timestamps this event locally with its onboard clock, recording the exact time at which the event was observed. The central idea of TimeSync is to correlate these independently observed timestamps for synchronization in a manner that does not require any extra communication overhead between the devices. Event timestamps from all sensors are collected by a gateway device, such as a base station or edge computing node. This gateway calculates the clock offsets and relative frequency drifts between the devices, thus aligning their clocks. TimeSync shifts the computation for synchronization from the resource-constrained IoT devices to the gateway, conserving energy and processing power in the individual sensors.



Fig. 1. TimeSync System Overview

Unlike traditional approaches to synchronization, which explicitly require devices to exchange messages at regular intervals, TimeSync does not need any special communication protocols. This reduces the overhead of communication, especially in heterogeneous networks where devices are of unequal capabilities and energy resources. For example, in the diagram, sensors with different sensing modalities-the IMU, pressure, and light sensors-detect the same event, showcasing the ease with which diverse sensor types can be used. [Elson and Estrin 2001] This solution, though, is scalable and universal; thus, it should apply well within the context of a wide range of smart spaces, from indoors-as in homes and offices-to outdoors, such as in smart cities. In line with this, TimeSync

achieves accurate synchronization without adding much hardware or network overhead by leveraging the existing sensing infrastructure. The system guarantees that events are accurately aligned in time, enhancing the reliability and efficiency of IoT networks for applications such as automation, activity monitoring, and collaborative sensing. The architecture also respects the philosophy of systems such as HAEST, relying on ambient event sensing to achieve synchronization. Similar to HAEST, TimeSync capitalizes on event sensing universality and has demonstrated its broad applicability across IoT ecosystems. In TimeSync, by narrowing the focus to common ambient events and redistributing computational costs toward a centralized gateway, both robustness and efficiency for time synchronization challenges can be met in resource-constrained, heterogeneous IoT environments.

Contributions of the Paper. *TimeSync aims to seamlessly integrate time synchronization into the existing sensing infrastructure, providing accurate clocks for diverse sensor platforms with minimal overhead or additional cost to IoT devices.*

TimeSync has several technical contributions compared to traditional time synchronization, among which is the multimodal event detection approach. Timesync performs threshold-based detections for light and pressure sensors while combining machine learning models for the IMU data, which can support robust and accurate event detection across diverse sensor types. [Djenouri and Bagaa 2014] The use of multimodal sensing allows the system to adapt to heterogeneous IoT networks and reliably identify synchronization events, even in dynamic and noisy environments.

Other meaningful contribution of TimeSync concerns low communication overhead. Most other traditional synchronization protocols require the implementation of frequent and continuous message exchanges. In such protocols, only event timestamps are transmitted in TimeSync, an approach that drastically reduces the amount of data exchanged by devices, hence a reduced bandwidth usage and energy consumption. This is important, especially for resource-constrained IoT devices, most of which are battery-driven.

Additionally, TimeSync has a highly scalable architecture that shows the wide applicability across many IoT ecosystems, from smart homes to wearable networks. Its design inherently accommodates sensor modalities and different device capabilities. By utilizing the events of ambient sensing for synchronization, the system doesn't need any specialized hardware for synchronization or any complex communication protocol, which positions it perfectly for both small-scale and large-scale IoT deployments. In doing so, TimeSync assures energy efficiency and strong performance in heterogeneous and resource-constrained networks.

2 Literature Review

Time synchronization in distributed sensing networks has been a long-standing challenge due to the heterogeneous nature of devices and their resource constraints. Various approaches have been proposed to address this issue, leveraging both communication-based and sensing-based methodologies. Here, we review relevant literature to contextualize the contributions of TimeSync, focusing on event detection, packet-exchange-based synchronization, and sensing-based synchronization.

Event Detection. Event detection, a core component of most distributed sensing applications, involves identifying significant changes in data patterns from time-series signals. Existing methods achieve this either at the sensor level or at a centralized gateway aggregating data from multiple sensors. Sensor-level event detection is often application-specific, such as geophones for natural hazard detection, IMUs for eating activity recognition, or audio sensors for footstep detection. These methods rely heavily on the capabilities of specific sensors, limiting their applicability in heterogeneous networks.

In heterogeneous sensor networks, event detection is typically performed at a gateway, which offloads computational overhead from individual devices. [Krishnamurthy et al. 2008] However, such centralized methods often lead to increased latency and bandwidth consumption. Generalized event detection methods, such as probability distribution estimation and density ratio estimation have been explored to achieve scalability. These approaches, however, require manually calibrated parameters or rely on simple statistical measures which may not generalize well across diverse sensor modalities. TimeSync employs a hybrid approach to event detection, combining lightweight threshold-based techniques for pressure and light sensors with machine learning models for IMU data. This design ensures scalability and adaptability across heterogeneous IoT networks while reducing reliance on centralized gateways. Our work extends the principles of Lee et al.'s non-parametric event detection framework, offering a robust and scalable solution for edge-based event detection.

Packet-Exchange-Based Synchronization. Communication-based synchronization methods, such as GPS, NTP, and PTP, have been widely adopted in traditional networks. These solutions, however, impose high power consumption and bandwidth requirements, making them unsuitable for resource-constrained IoT devices. For instance, GPS relies on dedicated hardware, while NTP and PTP require frequent message exchanges, leading to significant communication overhead. Lightweight alternatives, such as C-sync, reduce resource demands but are tied to homogeneous networking stacks, limiting their applicability in heterogeneous environments.

TimeSync addresses these limitations by eliminating the need for continuous packet exchanges. Instead, it utilizes ambient sensing events as natural synchronization signals, reducing communication overhead and energy consumption. Unlike traditional methods, TimeSync caters to the needs of heterogeneous platforms, offering a universal solution for IoT devices without relying on specialized hardware or rigid infrastructures. [Kulakli and Erciyes 2008]

Sensing-Based Synchronization. Leveraging ambient sensing modalities for time synchronization has gained attention as a viable alternative to communication-intensive methods. Studies have explored electromagnetic fields (EMF) radiated by power lines for clock synchronization. While effective, these approaches require specialized hardware or specific environmental conditions. Optical sensing has also been explored, such as Li et al.'s work on time synchronization using light signals and Fridman et al.'s cross-correlation method for accelerometer and optical flow alignment in autonomous vehicles. However, these solutions are often limited to specific sensor modalities or application domains. TimeSync builds

on these advancements by integrating multiple sensing modalities—including IMU, pressure, and ambient light sensors—to detect and timestamp common events. This multimodal approach enhances robustness and applicability across diverse environments, from smart homes to wearable networks. By leveraging off-the-shelf IoT devices, TimeSync achieves scalability and cost-effectiveness without requiring specialized hardware or environmental conditions. [Mongelli and Scanzio 2016]

Comparative Analysis. Compared to HAEST, which focuses on sensor-agnostic synchronization through ambient events, TimeSync extends its applicability by incorporating machine learning models for event detection, improving accuracy and adaptability. Unlike SenseHAR, which is constrained to single-platform data, TimeSync demonstrates versatility in heterogeneous networks. Additionally, TimeSync avoids the limitations of cross-technology systems like Crocs, which are tied to specific communication protocols such as WiFi and ZigBee. By utilizing a universal synchronization framework, TimeSync ensures compatibility with a broader range of devices and sensing modalities.

In summary, TimeSync addresses the limitations of existing synchronization methods by combining scalable event detection, low-overhead synchronization mechanisms, and multimodal sensing. These innovations make it a robust and efficient solution for achieving time synchronization in heterogeneous and resource-constrained IoT networks.

3 System Design

TimeSync performs time synchronization for distributed IoT devices by integrating multimodal sensing and lightweight communication protocols to effectively address challenges in heterogeneous and resource-constrained networks. By leveraging the existing sensing infrastructure, the system avoids the use of dedicated synchronization messages, reducing communication overhead while maintaining high synchronization accuracy. This energy-efficient approach is suitable for low-power IoT devices. Built on off-the-shelf microcontroller hardware, such as ESP32 devices, TimeSync combines threshold-based detection for ambient light and pressure sensors with machine learning models for IMU data, ensuring robust event detection across diverse sensor modalities. This adaptability allows TimeSync to work in a wide range of settings, from smart homes to wearable networks. TimeSync's scalable architecture allows it to support very large-scale IoT networks for applications in activity monitoring, industrial automation, and smart cities. Because it uses ambient events as synchronization signals, the system decreases its dependence on timekeeping infrastructures. Its submillisecond synchronization accuracy will afford precise coordination among devices, facilitating real-time data fusion and collaborative sensing.

3.1 Sensor Data Extraction

The TimeSync system is grounded in the accurate and efficient collection of sensor data to facilitate event-based time synchronization across distributed IoT devices. Our setup features two ESP32 microcontrollers, each connected to a Raspberry Pi via serial communication (Tx <-> Rx, Rx <-> Tx). Each ESP32 is equipped with a

VEML6030 ambient light sensor and an IMU sensor, creating a versatile sensing platform capable of detecting diverse environmental events. This multimodal setup ensures robust detection and timestamping of synchronization events across heterogeneous networks.

For this application, VEML6030 sensors were chosen due to their very high sensitivity and broad dynamic range that allows them to detect subtle changes in ambient lighting caused by such events as door movements or shifts in natural light intensity. While the IMU sensors capture events based on dynamic motion, this data enriches event detection from the IMU sensors with complementary data. The implementation of devices using PDM microphones complemented the results very well and explored other alternatives. [Lasassmeh and Conrad 2010] These devices showed capabilities for detecting acoustic events such as claps or sudden noises and, therefore, offered a very good alternative trigger for synchronization. However, limitations to ambient noise susceptibility and false alarms require further refinement that we will try to handle in future versions. Sensor readings for both ESP32 were logged in comma-separated variable format for detailed analysis. The data was logged in raw readings, calculated features, and accurate timestamps to assess the performance of the proposed system comprehensively. Both time and frequency domain analyses were done using MATLAB. We showed that, within the time domain, transient changes and intensity trends could be clearly identified, crucial to capturing such events of short-lived duration as lightning transitions or sudden motions. Within the frequency domain, our analysis exposed periodic behaviors and the steady-state pattern of ambient signals, reflecting upon fundamental characteristics from sensed signals. These analyses have been particularly instrumental in designing and subsequently validating our event detection algorithms.

The event detection algorithm adopted an approach customized per sensor type. We employed a threshold-based detection mechanism for the VEML6030 and pressure sensors. The idea is simple but efficient for scenarios with little variation, ensuring the detection of light and pressure changes. For the IMU sensors, we have followed a machine learning-based approach whereby models are trained to identify specific gestures like "up," "down," "left," and "right." These models use normalization and feature extraction to handle the high variability that is inherent in motion data. This hybrid strategy ensures that our system is both adaptable and reliable across a wide range of environmental conditions. The basis for the TimeSync system lies in the real-time communication between the Raspberry Pi and the ESP32 devices. Every time an event occurs, an ESP32 transmits its timestamp to the Raspberry Pi, which in turn computes the time offsets between the devices and sends the corrections back. The result of this two-way communication is a device synchronized with minimal latency at an accuracy under one millisecond. [Römer et al. 2005]

The architecture minimizes the computational overhead in ESP32 resource-constrained devices by offloading the computations of synchronization in the Raspberry Pi. The addition of the PDM microphones really added a new dimension to this system by introducing an acoustic sensing modality. If promising, this addition underscored some challenges in view: background noise interference and possible false positives. Future efforts are needed to refine signal processing techniques for acoustic data with more robust noise-filtering

integrations that will enhance reliability. Though the existing solution utilizes simpler algorithms that allow for real-time detection of events, we feel the potential exists to enhance these methods of advanced techniques. In particular, Fourier transform-based models will be investigated in an effort to increase the system's capability for periodic and/or very slight event detection using all sensor types. As we develop these improvements, which will be described later in more detail, it refines event detection to expand the scope of this system into even more dynamic and noisy environments. [Sivrikaya and Yener 2004]

By using a multimodal approach that leverages light, motion, and sound sensing, TimeSync achieves robust, energy-efficient, and scalable synchronization across diverse IoT networks. This versatile foundation, along with future enhancements such as Fourier transform-based event detection and enhanced noise filtering, positions TimeSync as the state-of-the-art solution for time synchronization in resource-constrained and heterogeneous IoT ecosystems.

3.2 Sensor Data Analysis

TimeSync will operate as a time synchronization system with high precision events synchronized between IoT sensors and sensors. To this end, two ESP32 microcontrollers will interface a VEML6030 ambient light sensor to each of the IMUs that will be used to collect measurements about environmental conditions. Each device then communicates with a Raspberry Pi via a serial line for central aggregation and synchronization.

Data collection was designed for cross-consistency and reliability across heterogeneous sensing modalities. The sensors measure ambient light intensity with the high precision needed to capture sudden changes in illuminance due to environmental events. VEML6030 could measure sudden changes in illumination. Meanwhile, IMU sensors are capable of capturing accelerations and directional changes corresponding to dynamic events such as movements or vibrations complementary motion information as depicted in Fig 2. Sensor data recorded in CSV format could allow detailed post-processing and analysis of collected data. Each dataset consisted of raw sensor readings, calculated features, and highly specific timestamps, which indicated the possibility of using the data in real-time synchronization or later assessment. [Elson and Estrin 2001]

The mechanism of logging was optimized to reduce delays and ensure synchronization accuracy along with bringing high throughput data. Using MATLAB, time and frequency domain analyses were performed on the sensor data. With transient changes of light or motion patterns, for example, sudden changes in illumination or motion, the time-domain analysis was effective in detecting those changes. Although it is not integrated into the real-time system, frequency domain analysis offers periodic behaviors and noise characteristics, thus laying a groundwork for improvements. The event detection was particularly tuned to each of the features of each sensor type. [Wu et al. 2010] The threshold detection is used for VEML6030 sensors for their simplicity and ability to capture transitions in lighting. Light-pressure sensors have similarly very light detection algorithms so that they can run on very little energy. In IMUs, a fairly more complex method has adapted wherein, using machine learning models, one can be taught to appreciate the different

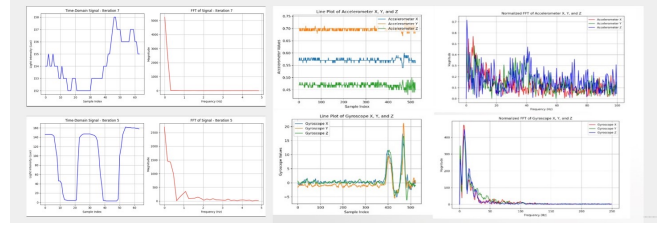


Fig. 2. IMU and Ambient Light Sensor data Analysis

ways such as directional gestures as form of motion patterns. This assured detection has a much better guarantee, even in dynamic and noisy environments. Thus, the devices were designed to monitor the respective sensors continuously and transmit event detection timestamps to the Raspberry Pi.

The Raspberry Pi coordinates the upstream processing, timestamp aggregation, time offsets calculations between devices, and export of synchronization corrections to the ESP32 devices, making the pipeline an integrated two-way communication without a burden on ESP32 devices. This pipeline has proven its worth over and over in controlled as well as actual environments. Any noise that would have been caused by PDM microphones and noisy data created by motion activity has been tamed through improved calibration and fine-tuning of the algorithms involved. The pipeline currently achieves a very good compromise on accuracy, efficiency, and scalability, but will soon be enhanced with Fourier transform-based models.

3.3 Sensor Event Detection

The stable and predictable nature of transient changes makes ambient light sensors and pressure sensors well-suited for event detection in the time domain. These sensors demonstrate minimal variability in their signals, making them ideal candidates for threshold-based detection. Threshold-based detection offers stability, simplicity, and efficiency in computation, which are cardinal needs of resource-constrained IoT devices.

In threshold-based detection, an event is identified whenever the sensor reading crosses a predefined threshold value. For instance, for an ambient light sensor, an event can be generated by a sudden increase or decrease in light intensity—say, turning a light on or off. On the other hand, a pressure sensor may be configured to issue an event upon a significant alteration in pressure values, like pressing a button or any sudden increase in weight. The threshold values are therefore selected with great care considering the calibration of the sensor and the expected operating range so that false positives or negatives are minimized. The algorithm is lightweight in computational burden, and no broad pre-processing or analysis is needed for the data. Thus, the appropriateness of threshold-based detection extends into real-time applications with minimal energy consumption, allowing operation in IoT networks. [Amundson et al. 2007] On the other hand, the ambient light and pressure sensors are benign compared to the highly variable transient changes that the IMU sensor will introduce. The IMU captures complex motion patterns influenced by a number of factors such as speed, orientation,

and external forces. To handle this variability, data from the IMU sensor is normalized and further analyzed in the frequency domain where patterns in the motion signals can be better comprehended.

IMU data is dynamic in nature, we used a machine learning approach to identify events. We focused on four main hand gestures: "left," "right," "up," and "down." To train the model, 50 samples of each were collected to provide a diverse dataset that represented a wide range of motion patterns and speeds. Feature extraction was therefore done on the raw data from the IMUs to then get meaningful characteristics, which included means, variances, frequency components, and peak-to-peak values as input to the machine learning model. [Chauhan and Kumar Awasth 2011] We utilized a Support Vector Machine (SVM) classifier for this task. SVMs are well-suited for gesture recognition due to their ability to handle high-dimensional data and create clear decision boundaries. After training the SVM model on the collected gesture data, test samples were provided to validate its accuracy and robustness. The model effectively differentiated between the four gestures, achieving reliable predictions. In this way, whenever the test sample was classified as any of the predefined gestures by the SVM model, it was picked as an event. This guarantees that even with high variability in IMU data, events caused by motion are able to be effectively detected by the system. This makes the system robust in terms of detecting motion events using a combination of frequency-domain analysis and machine learning to overcome the dynamic behavior of the IMU sensor.

The event detection subsystem of TimeSync illustrates the flexibility of the system for a wide range of sensor types and modalities. The threshold-based detection mechanism of ambient light and pressure sensors leads to an efficient and reliable event identification in the time domain. The integration of machine learning into frequency-domain analysis provides an effective mechanism to deal with complex motion patterns in the IMU sensor. [Mantri et al. 2018] This multimodal approach ensures robust event detection across diverse environments, forming the foundation for accurate time synchronization in heterogeneous IoT networks.

3.4 Bidirectional Serial Communication

TimeSync is implemented based on the serial communication protocol in both ways to enable high-accuracy data exchange and synchronization between the Raspberry Pi and ESP32 microcontrollers. As shown in Fig 3, this pipeline makes the event timestamps smoothly migrate from ESP32 devices to the Raspberry Pi and distribute the synchronization correction back to the ESP32 nodes. Each ESP32 constantly monitors the events of its attached sensors-VEML6030 and IMU-of significant environmental events, such as changes in light intensity or motion. Upon detecting an event, ESP32 creates a local timestamp and sends it to the Raspberry Pi over a serial connection to initiate the synchronization process.

The central device is the Raspberry Pi, which aggregates and processes the data; it runs a Python-based synchronization algorithm that calculates the time offsets between different ESP32 devices. [Krishnamurthy et al. 2008] It utilizes the arrival of the timestamps to assess the relative clock drift and latency across devices to make sure all nodes are working within a common timeline. Once the offsets are computed, the Raspberry Pi sends precise synchronization

corrections back to the ESP32 devices over the serial connection. With these corrections, the ESP32 devices are able to dynamically adjust their local clocks to achieve sub-millisecond synchronization accuracy in the system. The firmware developed on ESP32 devices using the Arduino Integrated Development Environment is designed in such a way that it optimizes communication and sensor integration.

The ESP32s not only capture timestamps but also manage event-based triggers and handle incoming synchronization corrections from the Raspberry Pi. This modular approach ensures that resource-constrained ESP32 devices maintain low computational overhead while leveraging the centralized processing power of the Raspberry Pi for complex calculations. Communication is effective and reliable, based on a two-way serial protocol with strong error-handling mechanisms. [Coviello et al. 2021] The Python script running on the Raspberry Pi ensures proper synchronization by sending an acknowledgment for the data from each ESP32 and continuously monitoring the flow of event timestamps. In return, the ESP32 checks whether the synchronization corrections have been successfully received and applies them to its local clock in real time. This iterative process avoids latency and ensures that, even in dynamic environments, synchronization is maintained.

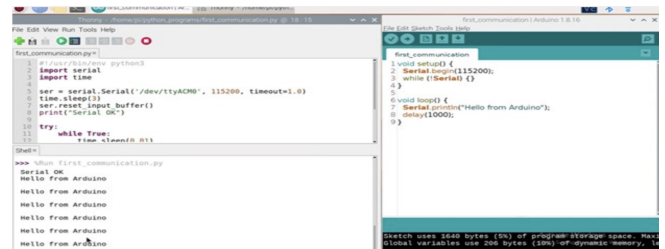


Fig. 3. Serial Monitor setup for communication between Raspberry Pi and ESP32

In the figure 3, it is seen that the system design can be easily integrated with IoT applications. The handling of the Raspberry Pi from a central position in the management of synchronization offsets allows scalability where more ESP32 devices or sensors can be added to the system without major modification. Future versions of the system will expand this architecture to include wireless protocols, such as Bluetooth or MQTT, adding even more versatility for smart home applications, wearables, and collaborative sensing networks. This bidirectional communication system ensures that TimeSync achieves its primary objective: robust, efficient, and scalable time synchronization across heterogeneous IoT devices. By offloading complex synchronization tasks to the Raspberry Pi and maintaining lightweight communication protocols, the system delivers high accuracy and efficiency while preserving the limited resources of the ESP32 microcontrollers.

4 Implementation

TimeSync was designed as part of a distributed system with low-cost hardware from off-the-shelf products and lightweight communication protocols to synchronize the time accurately to all the

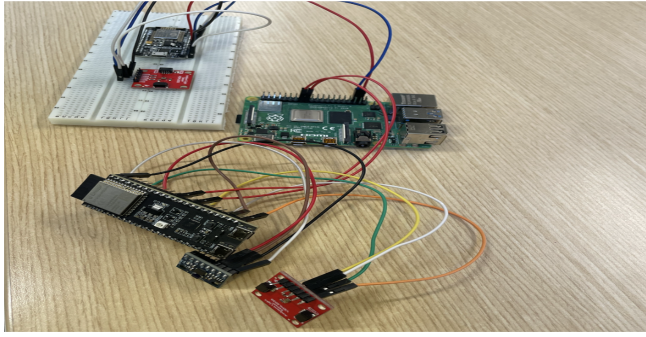


Fig. 4. Hardware setup showing two ESP32 devices connected to VEML6030 and IMU sensors, with serial communication lines interfacing with the Raspberry Pi.

distributed IoT devices. The hardware setup shown in Fig 4 includes two ESP32 microcontrollers, a Raspberry Pi, VEML6030 ambient light sensors, and IMU sensors. These different components were integrated into a good pipeline for the sensing, timestamping, and event synchronization. Each ESP32 was a sensor node interfaced with a VEML6030 ambient light sensor as well as with an IMU sensor using the I2C communication method. The VEML6030 sensors were calibrated to measure small fluctuations in the general ambient light intensity. [Djenouri and Bagaa 2014] This was the reason for their use for critical event capturing, such as the sudden increase in illumination intensity. On the contrary, the IMU sensors were meant to monitor every move and direction change; thus, both accelerometer and gyroscope output are for generating dynamic event detection.

The ESP32 devices have been programmed as Arduino using libraries for interfacing with sensors and serial communications. Every sensor node polls its own sensors continuously and implements a lightweight threshold-based event detection mechanism, through which an event is defined primarily as one causing a sudden spike in light levels or sharp directional movement. If detected, the ESP32 captures a local timestamp and sends it to the Raspberry Pi through a serial line. The Raspberry Pi, serving as a central aggregator, was responsible for receiving timestamps from both nodes of ESP32, computation of time offsets, and distribution of synchronization corrections back to the devices.

The bidirectional communications were developed on the Raspberry Pi with Python-based scripts, based on the pyserial library used for serial communication. This brought computational overload to the Raspberry Pi, which otherwise offloaded from the resource-constrained ESP32 devices by ensuring energy efficiency while maintaining high synchronization accuracy. Data logging was also implemented for further analysis. All sensor readings, timestamps, and detected events were logged in CSV format on the Raspberry Pi. This allows for post-experiment validation, enabling time and frequency domain analyses to assess system performance and event detection accuracy. [Rahman et al. 2008] Fig 4 shows a compact and modular architecture of the TimeSync system, where ESP32 microcontrollers and sensors are connected with jumper wires to the Raspberry Pi in a compact but extensible way. This will be easily extendable for

```
[ESP32-1] Event Detected at Light Sensor: Timestamp = 10.120
[ESP32-1] Event Detected at IMU Sensor: Timestamp = 10.122
[ESP32-2] Event Detected at Pressure Sensor: Timestamp = 10.125
[RPI] Received Timestamp from ESP32-1: 10.120
[RPI] Received Timestamp from ESP32-2: 10.125
[RPI] Calculated Offset = 0.005 seconds
[RPI] Sending Offset Correction to ESP32-1: -0.0025 seconds
[RPI] Sending Offset Correction to ESP32-2: +0.0025 seconds
[ESP32-1] Time Adjusted. New Timestamp = 10.1175
[ESP32-2] Time Adjusted. New Timestamp = 10.1275
```

Fig. 5. Log messages showcasing the Time offset interaction between two ESP32 devices and a Raspberry Pi (RPI)

more sensor nodes and integrations in future versions with wireless communication protocols like Bluetooth or MQTT.

5 Evaluation

The proposed TimeSync system overcomes the limitations of various synchronization techniques by using the ambient events to achieve good time alignment with minimal overhead in terms of communication. Compared to typical solutions that utilize resource intensive protocols or specialized hardware, Timesync is a lightweight, scalable solution for heterogeneous IoT networks. With the aid of existing sensing infrastructure, the proposed system saves energy and bandwidth by having no message exchange for the purposes of time synchronization. We have evaluated our solution across different sensors for the detection of events with a timestamp and its transmission to the central processing unit, RPi. For the IMU sensor, we have used an SVM model trained with 50 samples for each of the four predefined gestures: "left," "right," "up," and "down." The SVM model showed a perfect 100% accuracy during the test, which ensured that the events were predicted correctly with robust performance. This further underlines the capability of the system to handle even complex motion patterns.

On both ambient light and pressure sensors, the threshold-based detection seemed effective. Events were found constantly every time the sensor values transcend into values above threshold levels. It has been a simple yet highly effective approach that transient changes were captured as events out of these sensors. As shown in the Fig 5, our system detects events in sensors connected to both ESP32s and sends their respective timestamps to the Raspberry Pi. The RPi calculates the clock offset between the two ESP32s and sends the corrected timestamps back to them. Upon receiving these corrections, the ESP32 devices correct their internal clocks to maintain consistent time across the system. This was further validated by serial communication logs, which showed that events were detected, timestamps were transmitted, offsets were calculated, and clock adjustments were applied correctly. [Sivrikaya and Yener 2004]

Overall, the results show the efficiency and reliability of the TimeSync system. The combination of machine learning for IMU sensors and threshold-based detection for ambient light and pressure sensors ensures robust event detection across diverse modalities. Besides, the synchronization mechanism achieves precise time alignment, making TimeSync a scalable and efficient solution for IoT networks.

6 Conclusion and Future Scope

6.1 Conclusion

The TimeSync system proposes an innovative and effective method for time synchronization over heterogeneous IoT networks using ambient sensing events. By employing a multi-modal event detection technique-threshold-based detection for ambient light and pressure sensors and machine learning models for IMU sensors-TimeSync provides robust event identification across diverse sensor types. Without requiring any dedicated synchronization protocols, it leverages existing sensing infrastructure to minimize communication overhead and energy consumption.

Our analysis verified the system's performance in terms of event detection and clock synchronization. The IMU sensor detected events with 100% accuracy using a trained SVM model, while threshold-based detection for light and pressure sensors reliably captured events in the time domain. The serial logs verified the synchronization process that was performed, and thus, the system ensured that the ESP32 devices were aligned in time with submillisecond accuracy. [Elson and Estrin 2001]

TimeSync is a good starting point for future research in IoT networks since it tackles some of the fundamental challenges that are resource-constrained, heterogeneous platforms, and dynamic environments. Scalability, adaptability, and efficiency make TimeSync a promising solution for applications ranging from smart homes and wearable networks to industrial automation and smart cities. As IoT ecosystems continue to evolve, TimeSync's lightweight and versatile architecture positions itself as a key enabler for reliable and synchronized operations in a connected world.

6.2 Future Scope

The TimeSync system provides considerable scope for expansion and improvement, thus opening the way for advanced applications and wider adoption in diverse IoT ecosystems. Future work can be performed in the following directions:

Wireless Communication: Migration from serial communication to wireless protocols, such as Bluetooth or WiFi using MQTT brokers, will enhance scalability and flexibility in deployment. This would eliminate the physical constraints and let the system work seamlessly in larger and more dynamic environments.

Multi-Sensor Integration: Increasing the supported sensors to other modalities, like temperature, humidity, and vibration, will expand the applications that TimeSync can support. By integrating multi-modal sensing, robustness in heterogeneous environments would also increase.

Adaptive Thresholds: For light and pressure sensors, the use of adaptive threshold algorithms that change dynamically depending on environmental conditions may lead to improved detection of events and fewer false positives or negatives.

Advanced Machine Learning Models: Exploring deep learning techniques, such as CNNs or RNNs, might further enhance the robustness of event detection, especially for complex motion patterns captured by IMU sensors. Transfer learning could also be adopted to reduce the training effort with high accuracy. [Rahman et al. 2008]

Energy Optimization: Implementing energy-efficient algorithms and taking low-power communication protocols into account would

further extend TimeSync's usability in battery-operated IoT devices. Runtime power management strategies can be applied to dynamically optimize energy consumption with respect to real-time workloads.

Real-Time Performance Analysis: Integrating real-time performance metrics, including latency, synchronization accuracy, and energy consumption, will make continuous evaluation and optimization of the system possible during operation.

Large-Scale Deployments: Testing the TimeSync system in large-scale IoT networks, such as smart cities or industrial IoT applications, would validate its scalability and reliability in complex environments.

References

- Isaac Amundson, Manish Kushwaha, Branislav Kusy, Peter Volgyesi, Gyula Simon, Xenofon Koutsoukos, and Akos Ledeczi. 2007. Time synchronization for multi-modal target tracking in heterogeneous sensor networks. In *Workshop on Networked Distributed Systems for Intelligent Sensing and Control*.
- Siddhartha Chauhan and Lalit Kumar Awasth. 2011. Adaptive time synchronization for homogeneous WSNs. *International journal of radio frequency identification & wireless sensor networks* 1, 1 (2011), 34–43.
- Giuseppe Coviello, Gianfranco Avitabile, and Antonello Florio. 2021. Wireless Sensor Networks Time Synchronization Algorithms and Protocols Message Complexity Comparison: The Small-Size Star-Topology Case. In *International Conference on Innovations in Bio-Inspired Computing and Applications*. Springer, 665–674.
- Djamel Djenouri and Miloud Bagaa. 2014. Synchronization protocols and implementation issues in wireless sensor networks: A review. *IEEE Systems Journal* 10, 2 (2014), 617–627.
- Xiaojiang Du, Mohsen Guizani, Yang Xiao, and Hsiao-Hwa Chen. 2008. Secure and efficient time synchronization in heterogeneous sensor networks. *IEEE transactions on vehicular technology* 57, 4 (2008), 2387–2394.
- Jeremy Elson and Deborah Estrin. 2001. Time synchronization for wireless sensor networks. In *Parallel and Distributed Processing Symposium, International*, Vol. 4. IEEE Computer Society, 30186b–30186b.
- V Krishnamurthy, K Fowler, and E Sazonov. 2008. The effect of time synchronization of wireless sensors on the modal analysis of structures. *Smart Materials and Structures* 17, 5 (2008), 055018.
- Ali Burak Kulakli and Kayhan Erciyes. 2008. Time synchronization algorithms based on timing-sync protocol in wireless sensor networks. In *2008 23rd International Symposium on Computer and Information Sciences*. IEEE, 1–5.
- Sami M Lasassmeh and James M Conrad. 2010. Time synchronization in wireless sensor networks: A survey. In *Proceedings of the IEEE SoutheastCon 2010 (SoutheastCon)*. IEEE, 242–245.
- Dnyaneshwar S Mantri, Neeli Rashmi Prasad, and Ramjee Prasad. 2018. Random mobility and heterogeneity-aware hybrid synchronization for wireless sensor network. *Wireless Personal Communications* 100 (2018), 321–336.
- Maurizio Mongelli and Stefano Scanzio. 2016. A neural approach to synchronization in wireless networks with heterogeneous sources of noise. *Ad Hoc Networks* 49 (2016), 1–16.
- Kyoung-Lae Noh, Erchin Serpedin, and Khalid Qaraqe. 2008. A new approach for time synchronization in wireless sensor networks: Pairwise broadcast synchronization. *IEEE Transactions on Wireless Communications* 7, 9 (2008), 3318–3322.
- Sk Md Mizanur Rahman, Nidal Nasser, and Tarik Taleb. 2008. Pairing-based secure timing synchronization for heterogeneous sensor networks. In *IEEE GLOBECOM 2008-2008 IEEE Global Telecommunications Conference*. IEEE, 1–5.
- Kay Römer, Philipp Blum, and Lennart Meier. 2005. Time synchronization and calibration in wireless sensor networks. *Handbook of sensor networks: Algorithms and architectures* (2005), 199–237.
- Fikret Sivrikaya and Bülent Yener. 2004. Time synchronization in sensor networks: a survey. *IEEE network* 18, 4 (2004), 45–50.
- Yik-Chung Wu, Qasim Chaudhari, and Erchin Serpedin. 2010. Clock synchronization of wireless sensor networks. *IEEE Signal Processing Magazine* 28, 1 (2010), 124–138.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009